CÓDIGOCLARO · RUTA JAVASCRIPT

Tema 13 · Node.js y backend

Guía de estudio · DAM/DAW

process, módulos, variables de entorno y servidor HTTP.

1. Explicación

Node ejecuta JavaScript fuera del navegador y proporciona APIs como process y HTTP. process.argv contiene argumentos; process.env permite configurar opciones sin codificar secretos en el repositorio. Un servidor HTTP procesa solicitudes y debe validar método, ruta y datos recibidos. Guarda el ejemplo ESM como .mjs o ejecuta el proyecto con type: module.

2. Ejemplo

Código · javascript

```
// Guarda este archivo como servidor.mjs.
import {createServer} from "node:http";
const server = createServer((req,res) => {
  if (req.method === "GET" && req.url === "/salud") {
    res.writeHead(200, {"content-type":"text/plain; charset=utf-8"});
    res.end("ok");
  } else { res.writeHead(404); res.end("no encontrado"); }
});
server.listen(3000, () => console.log("Servidor en puerto 3000"));
```

Resultado esperado

```
Servidor en puerto 3000; GET /salud responde 200 y «ok».
```

3. Errores frecuentes

- Usar document en Node sin DOM.
- Guardar secretos en el código fuente.
- Ignorar método y ruta y contestar igual a toda petición.

4. Ejercicio

Responde 200/ok para GET /salud y 404 para rutas distintas.

Casos que debe superar

Entrada	Resultado esperado
GET /salud	200, ok
GET /desconocido	404
GET /salud/	404

5. Solución explicada

Código · javascript

```
// Guarda este archivo como servidor.mjs.
import {createServer} from "node:http";
const server = createServer((req,res) => {
  if (req.method === "GET" && req.url === "/salud") {
    res.writeHead(200, {"content-type":"text/plain; charset=utf-8"}); res.end("ok");
  } else { res.writeHead(404); res.end("no encontrado"); }
});
server.listen(Number(process.env.PORT) || 3000);
```

Otras soluciones válidas

- Usar una biblioteca de rutas cuando crezcan endpoints y middleware.
- Separar la lógica de dominio del adaptador HTTP.

6. Fuentes oficiales

Node.js — HTTP · Node.js — Process and environment